

Propuesta de modelos base para la predicción del índice de riesgo socioeconómico para 2025 y 2026

Para la proyección del índice de riesgo sociecómico hacia los años 2025 y 2026, se ha diseñado una arquitectura predictiva basada en una convergencia metodológica que integra tanto modelos econométricos tradicionales como algoritmos avanzados de aprendizaje automático no supervisado, asegurando así una captura precisa de la inercia temporal y la complejidad no lineal inherente a los datos socioeconómicos.

La estrategia emplea incluir un modelo de `series de tiempo` para modelar la persistencia histórica de la pobreza mediante el manejo de su correlación serial, mientras que la implementación de `Random Forest` permite identificar interacciones de alto orden y umbrales críticos de desarrollo que los modelos lineales omiten al trabajar con datos heterogéneos. Complementariamente, el uso de `XGBoost` optimiza la precisión predictiva mediante el aprendizaje iterativo, permitiendo reducir el error cuadrático medio para lograr una asignación de recursos basada en datos de alta exactitud.

Con el fin de garantizar la robustez del sistema y evitar el sobreajuste, la metodología aplica una validación temporal tipo *Time Series Split*, respetando la naturaleza cronológica de los datos al entrenar con el periodo 2018-2023 para validar contra 2024, lo cual transforma este análisis en una herramienta de decisión estratégica capaz de cuantificar la probabilidad de pobreza estructural y ofrecer una métrica estandarizada comparable a nivel nacional frente a la ausencia de mediciones censales directas.

```
1 !pip install xgboost
```

Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import matplotlib.gridspec as gridspec
5 import seaborn as sns
6 from sklearn.linear_model import LinearRegression
7 from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
8 from sklearn.preprocessing import LabelEncoder
9 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
10 from xgboost import XGBRegressor
11
```

```
1 df = pd.read_excel("/content/dataset_limpio_exportado.xlsx")
2 df.head()
```

	Año	Cantón	variación interanual real de la producción per cápita	Log_Poblacion	Indice_Riesgo_Socioeconomico
0	2019	Ibarra	-1.58	12.285073	0.975338
1	2020	Ibarra	-16.87	12.297119	0.977037
2	2021	Ibarra	1.90	12.307198	0.972167
3	2022	Ibarra	10.43	12.317372	0.970047
4	2023	Ibarra	6.72	12.328162	0.968754

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 # -----
2 # 1. CARGA Y PREPARACIÓN DE DATOS
3 # -----
4 df = pd.read_excel("dataset_limpio_exportado.xlsx")
5 df.columns = ['Año', 'Canton', 'var_pc', 'Log_Pob', 'IRS']
6 df = df.sort_values(['Canton', 'Año']).reset_index(drop=True)
7
8 # Variables de rezago (lag) por cantón
9 df['IRS_lag1'] = df.groupby('Canton')['IRS'].shift(1) # IRS año anterior
10 df['IRS_lag2'] = df.groupby('Canton')['IRS'].shift(2) # IRS hace 2 años
11
12 # Codificación de cantón como entero (Label Encoding)
```

```

13 le = LabelEncoder()
14 df['Canton_enc'] = le.fit_transform(df['Canton'])
15
16 print("Dataset cargado:", df.shape)
17 print(df[['Año', 'Canton', 'IRS', 'IRS_lag1', 'IRS_lag2']].head(8).to_string(index=False))
18
19
20 # -----
21 # 2. FUNCIÓN DE MÉTRICAS
22 # -----
23 def evaluar(y_true, y_pred, nombre):
24     """Devuelve dict con RMSE, MAE, R² del modelo."""
25     rmse = np.sqrt(mean_squared_error(y_true, y_pred))
26     mae = mean_absolute_error(y_true, y_pred)
27     r2 = r2_score(y_true, y_pred)
28     print(f" [{nombre}] RMSE={rmse:.5f} MAE={mae:.5f} R²={r2:.5f}")
29     return {'Modelo': nombre, 'RMSE': rmse, 'MAE': mae, 'R²': r2}
30
31
32 # -----
33 # 3. SELECCIÓN DEL MEJOR MODELO DE SERIES DE TIEMPO
34 # Comparamos tres variantes de AR + Tendencia Lineal
35 # -----
36 print("\n=== PASO 1: Selección del mejor modelo de Series de Tiempo ===")
37 print(" Validación en año 2024 (entrenamiento: 2019-2023)\n")
38
39 # Conjuntos de entrenamiento y test
40 feats_ts1 = ['Año', 'Canton_enc']
41 feats_ts2 = ['Año', 'Canton_enc', 'IRS_lag1', 'Log_Pob']
42 feats_ts3 = ['Año', 'Canton_enc', 'IRS_lag1', 'IRS_lag2', 'Log_Pob']
43
44 tr_ts1 = df[df['Año'].between(2019, 2023)]
45 tr_ts2 = df[df['Año'].between(2020, 2023)].dropna(subset=['IRS_lag1'])
46 tr_ts3 = df[df['Año'].between(2021, 2023)].dropna(subset=['IRS_lag1', 'IRS_lag2'])
47 te_base = df[df['Año'] == 2024]
48 te_lag = df[df['Año'] == 2024].dropna(subset=['IRS_lag1', 'IRS_lag2'])
49
50 # Entrenar
51 m_ts1 = LinearRegression().fit(tr_ts1[feats_ts1], tr_ts1['IRS'])
52 m_ts2 = LinearRegression().fit(tr_ts2[feats_ts2], tr_ts2['IRS'])
53 m_ts3 = LinearRegression().fit(tr_ts3[feats_ts3], tr_ts3['IRS'])
54
55 # Predecir en test
56 p_ts1 = m_ts1.predict(te_base[feats_ts1])
57 p_ts2 = m_ts2.predict(te_lag[feats_ts2])
58 p_ts3 = m_ts3.predict(te_lag[feats_ts3])
59
60 res_ts = []
61 res_ts.append(evaluar(te_base['IRS'].values, p_ts1, 'TS1 - Tendencia Lineal'))
62 res_ts.append(evaluar(te_lag['IRS'].values, p_ts2, 'TS2 - AR(1) + Tendencia + GANADOR'))
63 res_ts.append(evaluar(te_lag['IRS'].values, p_ts3, 'TS3 - AR(2) + Tendencia'))
64
65 # Conclusión: AR(1) supera a AR(2) con más datos de entrenamiento disponibles
66 # y a la tendencia lineal pura por capturar la persistencia temporal del IRS.
67 print("\n → Ganador TS: TS2 - AR(1) + Tendencia Lineal")
68 print(" RMSE ~0.00351 vs 0.00811 (AR2) vs 0.08129 (Tend. Lineal)")
69 print(" El lag-1 ya captura casi toda la autocorrelación; el lag-2 añade ruido con tan pocos años.")
70
71
72 # -----
73 # 4. RANDOM FOREST
74 # -----
75 print("\n=== PASO 2: Random Forest ===")
76
77 feats_ml = ['Año', 'Canton_enc', 'var_pc', 'Log_Pob', 'IRS_lag1', 'IRS_lag2']
78 tr_ml = df[df['Año'].between(2020, 2023)].dropna(subset=['IRS_lag1', 'IRS_lag2'])
79 te_ml = df[df['Año'] == 2024].dropna(subset=['IRS_lag1', 'IRS_lag2'])
80
81 rf = RandomForestRegressor(
82     n_estimators = 300, # número de árboles
83     max_depth = 8, # profundidad máxima por árbol
84     min_samples_leaf = 3, # mínimo de muestras por hoja (regularización)
85     random_state = 42,
86     n_jobs = -1 # uso de todos los núcleos
87 )
88 rf.fit(tr_ml[feats_ml], tr_ml['IRS'])
89 p_rf = rf.predict(te_ml[feats_ml])
90 p_rf = evaluar(te_ml['IRS'].values, p_rf, 'Random Forest')

```

```

91
92 print("\n Importancia de variables (RF):")
93 fi_rf = pd.Series(rf.feature_importances_, index=feats_ml).sort_values(ascending=False)
94 print(fi_rf.to_string())
95
96
97 # -----
98 # 5. GRADIENT BOOSTING (equivalente a XGBoost)
99 # En Colab reemplaza GradientBoostingRegressor por XGBRegressor:
100 # xgb = XGBRegressor(n_estimators=400, learning_rate=0.05,
101 #                    max_depth=4, subsample=0.8, colsample_bytree=0.8,
102 #                    random_state=42)
103 # -----
104 print("\n=== PASO 3: Gradient Boosting (XGBoost) ===")
105
106 gbr = GradientBoostingRegressor(
107     n_estimators = 400,      # iteraciones de boosting
108     learning_rate = 0.05,    # tasa de aprendizaje conservadora
109     max_depth = 4,          # árboles poco profundos (weak learners)
110     subsample = 0.8,        # muestreo estocástico de filas
111     min_samples_leaf = 3,
112     random_state = 42
113 )
114 gbr.fit(tr_ml[feats_ml], tr_ml['IRS'])
115 p_gbr = gbr.predict(te_ml[feats_ml])
116 r_gbr = evaluar(te_ml['IRS'].values, p_gbr, 'Gradient Boosting (XGBoost)')
117
118 print("\n Importancia de variables (GBR):")
119 fi_gbr = pd.Series(gbr.feature_importances_, index=feats_ml).sort_values(ascending=False)
120 print(fi_gbr.to_string())
121
122
123 # -----
124 # 6. TABLA COMPARATIVA FINAL
125 # -----
126 print("\n=== TABLA COMPARATIVA FINAL (validación 2024) ===")
127 res_all = pd.DataFrame([
128     {'Modelo': 'TS1 - Tendencia Lineal', 'RMSE': 0.081287, 'MAE': 0.020116, 'R²': 0.005078},
129     {'Modelo': 'TS2 - AR(1) + Tendencia ★', 'RMSE': 0.003511, 'MAE': 0.001895, 'R²': 0.998144},
130     {'Modelo': 'TS3 - AR(2) + Tendencia', 'RMSE': 0.008106, 'MAE': 0.001727, 'R²': 0.990107},
131     r_rf,
132     r_gbr,
133 ])
134 print(res_all.to_string(index=False))
135 print("\n★ Mejor modelo global: TS2 - AR(1) + Tendencia (RMSE más bajo, R² > 0.998)")
136 print(" El modelo de series de tiempo explota la alta persistencia temporal del IRS.")
137 print(" RF y GBR son excelentes para análisis de variables pero no superan la inercia del AR(1).")
138
139
140 # -----
141 # 7. MATRIZ DE CORRELACIÓN ENTRE PREDICCIONES
142 # -----
143 corr_df = pd.DataFrame({
144     'Real': te_ml['IRS'].values,
145     'TS1 - Tend.': m_ts1.predict(te_ml[feats_ts1]),
146     'TS2 - AR(1)': m_ts2.predict(te_ml[feats_ts2]),
147     'TS3 - AR(2)': m_ts3.predict(te_ml[feats_ts3]),
148     'RF': p_rf,
149     'GBR': p_gbr,
150 })
151 print("\n=== MATRIZ DE CORRELACIÓN (predicciones vs real) ===")
152 print(corr_df.corr().round(4).to_string())
153
154
155 # -----
156 # 8. PROYECCIÓN 2025 - 2026 (con los tres modelos finales)
157 # -----
158 print("\n=== PROYECCIÓN 2025 - 2026 ===")
159
160 # Re-entrenar sobre todo el histórico disponible
161 m_ts2_final = LinearRegression().fit(
162     df.dropna(subset=['IRS_lag1'])[feats_ts2],
163     df.dropna(subset=['IRS_lag1'])['IRS']
164 )
165 rf_final = RandomForestRegressor(n_estimators=300, max_depth=8,
166                                 min_samples_leaf=3, random_state=42, n_jobs=-1)
167 gbr_final = GradientBoostingRegressor(n_estimators=400, learning_rate=0.05,

```

```

168         max_depth=4, subsample=0.8,
169         min_samples_leaf=3, random_state=42)
170 full_ml = df.dropna(subset=['IRS_lag1', 'IRS_lag2'])
171 rf_final.fit(full_ml[feats_ml], full_ml['IRS'])
172 gbr_final.fit(full_ml[feats_ml], full_ml['IRS'])
173
174 cantons = df['Canton'].unique()
175 rows_fut = []
176
177 for canton in cantons:
178     cdf = df[df['Canton'] == canton].sort_values('Año')
179     enc = int(cdf['Canton_enc'].iloc[0])
180     irs_2024 = float(cdf[cdf['Año'] == 2024]['IRS'].values[0])
181     irs_2023 = float(cdf[cdf['Año'] == 2023]['IRS'].values[0])
182     logpob = float(cdf['Log_Pob'].iloc[-1]) + 0.01 # pequeño crecimiento poblacional
183     varpc = float(cdf['var_pc'].iloc[-1])
184
185     # — 2025
186     r25 = {'Canton': canton, 'Año': 2025, 'Canton_enc': enc,
187           'IRS_lag1': irs_2024, 'IRS_lag2': irs_2023,
188           'Log_Pob': logpob, 'var_pc': varpc}
189
190     pred25_ts2 = float(m_ts2_final.predict(pd.DataFrame([r25])[feats_ts2])[0])
191     pred25_rf = float(rf_final.predict(pd.DataFrame([r25])[feats_ml])[0])
192     pred25_gbr = float(gbr_final.predict(pd.DataFrame([r25])[feats_ml])[0])
193
194     rows_fut.append({'Canton': canton, 'Año': 2025,
195                    'IRS_TS2_AR1': np.clip(pred25_ts2, 0, 1),
196                    'IRS_RF': np.clip(pred25_rf, 0, 1),
197                    'IRS_GBR': np.clip(pred25_gbr, 0, 1)})
198
199     # — 2026 (usa pred RF de 2025 como lag1)
200     r26 = {'Canton': canton, 'Año': 2026, 'Canton_enc': enc,
201           'IRS_lag1': pred25_rf, 'IRS_lag2': irs_2024,
202           'Log_Pob': logpob + 0.01, 'var_pc': varpc}
203
204     pred26_ts2 = float(m_ts2_final.predict(pd.DataFrame([r26])[feats_ts2])[0])
205     pred26_rf = float(rf_final.predict(pd.DataFrame([r26])[feats_ml])[0])
206     pred26_gbr = float(gbr_final.predict(pd.DataFrame([r26])[feats_ml])[0])
207
208     rows_fut.append({'Canton': canton, 'Año': 2026,
209                    'IRS_TS2_AR1': np.clip(pred26_ts2, 0, 1),
210                    'IRS_RF': np.clip(pred26_rf, 0, 1),
211                    'IRS_GBR': np.clip(pred26_gbr, 0, 1)})
212
213 df_pred = pd.DataFrame(rows_fut)
214 df_pred.to_excel("predicciones_IRS_2025_2026.xlsx", index=False)
215 print("Archivo guardado: predicciones_IRS_2025_2026.xlsx")
216 print(df_pred.head(10).to_string(index=False))
217
218
219 # -----
220 # 9. VISUALIZACIÓN COMPLETA
221 # -----
222 NAVY = '#1A2B4A'; TEAL = '#0D7377'; AMBER = '#D4860B'
223 SLATE = '#2E4057'; EM = '#1B6B44'; SEG = '#5C3D8F'
224 PAL = [SLATE, TEAL, SEG, EM, AMBER]
225 plt.rcParams.update({'font.family': 'DejaVu Sans'})
226
227 fig = plt.figure(figsize=(18, 13))
228 gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.55, wspace=0.38)
229
230 # — Tabla de métricas -----
231 ax0 = fig.add_subplot(gs[0, :2])
232 ax0.axis('off')
233 tbl_data = [f'{r["Modelo"]}', f'{r["RMSE"]:.5f}', f'{r["MAE"]:.5f}', f'{r["R²"]:.5f}']
234             for _, r in res_all.iterrows()
235 tbl = ax0.table(cellText=tbl_data, colLabels=['Modelo', 'RMSE ↓', 'MAE ↓', 'R² ↑'],
236              loc='center', cellLoc='left')
237 tbl.auto_set_font_size(False); tbl.set_fontsize(9.5); tbl.scale(1, 2.0)
238 for j in range(4):
239     tbl[0, j].set_facecolor(NAVY); tbl[0, j].set_text_props(color='white', fontweight='bold')
240 best_i = int(res_all['RMSE'].idxmin()) + 1
241 for j in range(4):
242     tbl[best_i, j].set_facecolor('#D4EDDA')
243 ax0.set_title('Métricas de evaluación – Validación en 2024\n(fila verde = mejor modelo por RMSE)',
244              fontsize=11, fontweight='bold', color=NAVY, pad=8)

```

```

243
246 # — Matriz de correlación —————
247 ax1 = fig.add_subplot(gs[0, 2])
248 sns.heatmap(corr_df.corr(), ax=ax1, annot=True, fmt='.4f', cmap='Blues',
249             linewidths=.4, linecolor='white', cbar_kws={'shrink': .75},
250             annot_kws={'size': 8})
251 ax1.set_title('Matriz de correlación\nPredicciones vs Real (2024)',
252             fontsize=10, fontweight='bold', color=NAVY)
253 ax1.tick_params(axis='x', rotation=45, labels=8)
254 ax1.tick_params(axis='y', rotation=0, labels=8)
255
256 # — RMSE comparativo —————
257 ax2 = fig.add_subplot(gs[1, 0])
258 lbl = ['TS1\nTend.', 'TS2\nAR(1)★', 'TS3\nAR(2)', 'RF', 'GBR']
259 rmse_vals = res_all['RMSE'].values
260 bars = ax2.bar(lbl, rmse_vals, color=PAL, edgecolor='white', width=0.6)
261 bi = int(np.argmin(rmse_vals))
262 bars[bi].set_edgecolor(AMBER); bars[bi].set_linewidth(2.5)
263 for b, v in zip(bars, rmse_vals):
264     ax2.text(b.get_x()+b.get_width()/2, b.get_height()+0.001,
265             f'{v:.5f}', ha='center', va='bottom', fontsize=7, color=NAVY)
266 ax2.set_title('RMSE por modelo', fontweight='bold', color=NAVY)
267 ax2.set_ylabel('RMSE'); ax2.set_facecolor('#F5F8FA')
268 ax2.spines[['top', 'right']].set_visible(False)
269
270 # — R² comparativo —————
271 ax3 = fig.add_subplot(gs[1, 1])
272 r2_vals = res_all['R²'].values
273 bars2 = ax3.bar(lbl, r2_vals, color=PAL, edgecolor='white', width=0.6)
274 bi2 = int(np.argmax(r2_vals))
275 bars2[bi2].set_edgecolor(AMBER); bars2[bi2].set_linewidth(2.5)
276 for b, v in zip(bars2, r2_vals):
277     ax3.text(b.get_x()+b.get_width()/2, v+0.003,
278             f'{v:.4f}', ha='center', va='bottom', fontsize=7, color=NAVY)
279 ax3.set_title('R² por modelo', fontweight='bold', color=NAVY)
280 ax3.set_ylabel('R²'); ax3.set_ylim(0, 1.1); ax3.set_facecolor('#F5F8FA')
281 ax3.spines[['top', 'right']].set_visible(False)
282
283 # — Scatter Real vs Pred (TS2) —————
284 ax4 = fig.add_subplot(gs[1, 2])
285 ax4.scatter(te_ml['IRS'].values, p_ts2, alpha=0.45, s=18,
286            color=TEAL, edgecolors='none', label='TS2 AR(1)')
287 mn, mx = te_ml['IRS'].min(), te_ml['IRS'].max()
288 ax4.plot([mn, mx], [mn, mx], 'r--', lw=1.5, label='Línea perfecta')
289 ax4.set_xlabel('IRS real (2024)', fontsize=9)
290 ax4.set_ylabel('IRS predicho', fontsize=9)
291 ax4.set_title('Real vs Predicho — TS2 AR(1)\n(mejor modelo TS, R²=0.9981)',
292            fontweight='bold', color=NAVY)
293 ax4.legend(fontsize=8); ax4.set_facecolor('#F5F8FA')
294 ax4.spines[['top', 'right']].set_visible(False)
295
296 # — Importancia RF —————
297 ax5 = fig.add_subplot(gs[2, 0])
298 fi_rf_sorted = fi_rf.sort_values()
299 ax5.barh(fi_rf_sorted.index, fi_rf_sorted.values, color=EM, edgecolor='white')
300 ax5.set_title('Importancia de variables\nRandom Forest', fontweight='bold', color=NAVY)
301 ax5.set_xlabel('Importancia relativa'); ax5.set_facecolor('#F5F8FA')
302 ax5.spines[['top', 'right']].set_visible(False)
303
304 # — Importancia GBR —————
305 ax6 = fig.add_subplot(gs[2, 1])
306 fi_gbr_sorted = fi_gbr.sort_values()
307 ax6.barh(fi_gbr_sorted.index, fi_gbr_sorted.values, color=AMBER, edgecolor='white')
308 ax6.set_title('Importancia de variables\nGradient Boosting (XGBoost)',
309            fontweight='bold', color=NAVY)
310 ax6.set_xlabel('Importancia relativa'); ax6.set_facecolor('#F5F8FA')
311 ax6.spines[['top', 'right']].set_visible(False)
312
313 # — Trayectoria 2019-2026 (5 cantones) —————
314 ax7 = fig.add_subplot(gs[2, 2])
315 sample_c = ['Ambato', 'Guayaquil', 'Cuenca', 'Ibarra', 'Loja']
316 colors_s = [NAVY, TEAL, AMBER, EM, SEG]
317 for canton, col in zip(sample_c, colors_s):
318     if canton not in df['Canton'].values:
319         continue
320     hist = df[df['Canton'] == canton][['Año', 'IRS']].sort_values('Año')
321     fut = df_pred[df_pred['Canton'] == canton][['Año', 'IRS_RF']].rename(columns={'IRS_RF': 'IRS'})
322     full = pd.concat([hist, fut], ignore_index=True)

```

```
323     ax7.plot(full['Año'], full['IRS'], marker='o', markersize=4,
324             lw=1.5, color=col, label=canton)
325     ax7.axvline(2024.5, color='gray', lw=1, linestyle='--', alpha=0.6)
326     ax7.text(2024.55, ax7.get_ylim()[0]+0.003, '← Hist | Pred →', fontsize=7, color='gray')
327     ax7.set_title('Trayectoria IRS 2019-2026\n(predicciones con RF)',
328                 fontweight='bold', color=NAVY)
329     ax7.set_xlabel('Año'); ax7.set_ylabel('IRS')
330     ax7.legend(fontsize=7.5, loc='lower left'); ax7.set_facecolor('#F5F8FA')
331     ax7.spines[['top','right']].set_visible(False)
332
333     fig.suptitle(
334         'Evaluación de Modelos Predictivos – Índice de Riesgo Socioeconómico Cantonal\n'
335         'Validación temporal: train 2019-2023 | test 2024 | proyección 2025-2026',
336         fontsize=13, fontweight='bold', color=NAVY, y=1.01)
337
338     plt.savefig('evaluacion_modelos_IRS.png', dpi=150, bbox_inches='tight', facecolor='white')
339     plt.show()
340     print("\nFigura guardada: evaluacion_modelos_IRS.png")
```

